

Your database learning guide

PostgreSQL from zero, using your Restaurant QR Ordering SaaS. This guide explains the commands you have already typed and the database tools you will use as the project grows. It is a learning reference, not a complete production database implementation or a catalog of every PostgreSQL command.

You do not need to memorize commands. Read what a command means, predict its result, type it yourself, and compare the result with your prediction. Learn one lesson at a time.

How to use this guide

Lessons 2-14 rebuild your current understanding. Lessons 15-22 introduce changes, transactions, performance and operations. Lesson 23 explains Node. Lessons 25-36 apply SQL to your full product scope; lesson 37 is the command reference.

Use `restaurant_ordering_practice` for exercises. Your working project database is `restaurant_ordering_dev`. The examples here create a fresh practice schema, so its category IDs may differ from the existing project. Always inspect IDs before using them.

The structure you are learning

```
PostgreSQL server on your Mac
  restaurant_ordering_dev database
    restaurants table
    menu_categories table
    menu_items table
```

One restaurant has many categories.
One category has many menu items.

First target: explain why Tea has a category ID, why that category has a restaurant ID, and how a query follows those relationships. Understanding this is more useful than remembering SQL by sight.

Prepared for your local PostgreSQL 18 learning setup. Commands are examples to type deliberately; the guide does not change your project database.

Server, database, table and row

The server is the running PostgreSQL program. It receives requests and saves data. A database is a named collection inside that server. A table holds records of one kind. Each row is one record; each column describes one part of it.

Word	Example in your project
Server	The PostgreSQL service running on your Mac.
Database	restaurant_ordering_dev contains this project data.
Table	restaurants stores restaurant records.
Row	The Student cafe record.
Column	name, area or cuisine.
Schema	public is a namespace inside a database; your tables currently live there.

A table definition describes allowed columns, types and rules. Its rows are the actual saved values. Creating a table does not automatically fill it with rows.

```
id | name          | area      | cuisine
1  | Student cafe | Sitapura | Cafe
2  | Oslo         | Jagatpura| Indian
```

The first row has an ID, a name, an area and a cuisine. Its ID is 1. The name is a value in a column; it is not a table or another database.

Both psql and the Express API can talk to the same PostgreSQL server. They do not hold separate copies of the restaurant table.

Try it yourself

Draw a rectangle for the restaurants table. Label its four columns, then draw one row for a made-up restaurant. Explain the difference between the table definition and that row.

Shell commands and SQL prompts

The Mac shell and PostgreSQL understand different languages. The prompt tells you where you are. This was the reason CREATE TABLE failed with a zsh error in your screenshot.

Prompt	What belongs there
backend %	Shell commands: run programs, change folders, start services.
database_name=#	SQL statements and psql backslash commands.
database_name-#	An SQL statement is unfinished; psql is waiting.
database_name(#	An opening parenthesis has not been closed.

Run these in the Mac terminal

```
brew install postgresql@18
brew services start postgresql@18
/opt/homebrew/opt/postgresql@18/bin/pg_isready
```

brew install installs software. brew services start starts the background server and registers it to start at login. pg_isready checks whether a server is accepting connections. It does not test a particular table.

```
/opt/homebrew/opt/postgresql@18/bin/psql -d postgres
```

The path selects the psql program. The -d option chooses a database. postgres is an existing database used to get started. Your Apple Silicon Homebrew paths are shown here; another machine may have different paths.

Ctrl+C cancels unfinished input in psql. It does not delete the table. Do not type the prompt itself as part of a command.

Try it yourself

Open psql and identify its prompt. Exit with \q. Identify the shell prompt after exiting.

Create a practice database

The database you already created for the project is `restaurant_ordering_dev`. Use a separate database while learning so exercises do not change your project records.

Inside psql, connected to postgres

```
CREATE DATABASE restaurant_ordering_practice;  
  
\c restaurant_ordering_practice  
  
SELECT current_database(), current_user;
```

`CREATE DATABASE` creates a named database. The semicolon ends that SQL statement. If it already exists, PostgreSQL reports an error; connect to it instead of repeating the creation command.

`\c` reconnects psql to the selected database. `SELECT` asks for values. `current_database()` tells you your database; `current_user` tells you your PostgreSQL role. A role is a database identity, not the customer entering their display name.

Look around

```
\l  
\dt  
\conninfo  
\q
```

`\l` lists databases. `\dt` lists visible tables. `\conninfo` shows connection details. `\q` exits psql without stopping the server. Backslash commands generally do not end with a semicolon.

You can open the practice database directly from the shell using the same psql program with `-d restaurant_ordering_practice`.

Try it yourself

Check `current_database()` before each practice session. You should see `restaurant_ordering_practice`, not the project database.

SQL punctuation and values

SQL reads instructions in statements. Line breaks make long statements easier for people to read; the semicolon tells psql that the statement is complete.

```
SELECT 'Tea' AS item_name, 3000 AS price_paise;
```

SELECT asks PostgreSQL to return values. Tea is text, so it uses single quotes. 3000 is a number, so it has no quotes. AS gives a result column a readable label. A comma separates the two requested columns.

Syntax	Meaning
'Tea'	A text value.
name	A column or other SQL identifier.
NULL	A missing or unknown value; not the text string NULL.
TRUE / FALSE	Boolean values.
(...)	A group, column list, value list or function argument list.
-- comment	Notes ignored by SQL until the end of the line.
;	Ends the SQL statement.

SQL keywords such as SELECT are case-insensitive. Unquoted identifiers are folded to lowercase. Text comparisons can depend on type and collation; use consistent spelling for now. Use lowercase snake_case for table and column names.

```
SELECT 'Chef''s special' AS item_name;
```

Inside SQL text, two single quotes represent one literal apostrophe. In API queries, send user values as parameters rather than constructing quoted SQL yourself.

Try it yourself

Return your name and a made-up price using SELECT, with clear labels. Predict the two output column names first.

Create your first table

```
CREATE TABLE restaurants (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name TEXT NOT NULL,  
  area TEXT NOT NULL,  
  cuisine TEXT NOT NULL  
);
```

Run this once in the practice database. CREATE TABLE names the new table. Each line inside parentheses defines a column. Commas separate definitions; the last definition has no comma after it.

id BIGINT stores a whole number. GENERATED ALWAYS AS IDENTITY makes PostgreSQL supply IDs. PRIMARY KEY makes each ID unique and non-null. An identity definition alone is not a uniqueness rule; PRIMARY KEY supplies that rule.

name TEXT NOT NULL defines a text column that cannot contain NULL. area and cuisine follow the same pattern. TEXT is a data type; NOT NULL is a constraint, meaning an enforced rule.

```
\d restaurants
```

This psql command displays the table definition, including column types, defaults and constraints. It does not show restaurant rows.

Identity values can have gaps after failed inserts, rollbacks or deletions. Do not use IDs to count records or assume the next ID. A BIGINT has a maximum; it is not an unlimited number.

Try it yourself

Inspect the table. Explain which part generates an ID and which part guarantees that IDs are unique. Keep your existing project table; do not recreate it there.

Types and rules protect your data

Type or rule	Meaning and example
TEXT	Text such as a restaurant name.
INTEGER	Whole numbers; suitable for bounded paise prices.
BIGINT	Larger whole numbers; your generated IDs use this.
BOOLEAN	TRUE or FALSE; menu availability.
TIMESTAMPTZ	An instant in time; useful for future order timestamps.
NOT NULL	Reject a missing value. An empty string is still a value.
DEFAULT	Supply a value when an insert omits the column.
PRIMARY KEY	Identify a row uniquely.
FOREIGN KEY	Require a relationship to an existing referenced row.
UNIQUE	Prevent duplicate values or duplicate combinations.
CHECK	Enforce a condition for the saved row.

A default does not replace an explicit NULL with a valid value. A CHECK can allow NULL unless NOT NULL is also present. Your price column uses both rules.

```
SELECT 5000 AS price_paise, 5000 / 100.0 AS price_rupees;
```

Store 5000 paise for INR 50.00. The API computes totals from trusted prices. The decimal divisor here only illustrates display conversion; stored prices and order amounts stay in integer paise.

Later we will store order times using TIMESTAMPTZ and keep restaurant time zones separately. Its display changes with the connection time zone; it does not retain the original zone name.

Try it yourself

For a menu item, explain why name is TEXT, price is INTEGER and availability is BOOLEAN. What is the difference between NULL and an empty name?

Insert restaurant rows

```
INSERT INTO restaurants (name, area, cuisine)
VALUES
  ('Student cafe', 'Sitapura', 'Cafe'),
  ('Oslo', 'Jagatpura', 'Indian')
RETURNING id, name;
```

INSERT INTO chooses the table. The column list sets the order of the incoming values. VALUES introduces the data. Each parenthesized group creates one row, and commas separate the rows.

In the first row, Student cafe goes into name, Sitapura into area, and Cafe into cuisine. We omit id because PostgreSQL generates it.

RETURNING shows values from the newly inserted rows. This is useful when you need the generated IDs. Without RETURNING, psql shows a command summary such as INSERT 0 2: two rows were inserted.

```
SELECT id, name, area, cuisine
FROM restaurants
ORDER BY id;
```

SELECT reads columns. FROM names their table. ORDER BY id sorts ascending by default. Use DESC for largest first. Without ORDER BY, row order is not guaranteed.

Repeated INSERT statements create additional rows unless a constraint stops them. The current restaurants table does not require unique names, so rerunning these inserts can create duplicates.

Try it yourself

Insert one more made-up restaurant. Use RETURNING to learn its ID. Read all restaurants and identify exactly which row you added.

Read and filter rows

```
SELECT id, name, area
FROM restaurants
WHERE id = 1;
```

```
SELECT id, name
FROM restaurants
WHERE cuisine = 'Cafe'
ORDER BY name;
```

WHERE keeps rows that satisfy a condition. `id = 1` compares the ID with a number. `cuisine = Cafe` compares text with a quoted text value. SELECT names the columns you want back.

Combining conditions

```
SELECT id, name
FROM restaurants
WHERE cuisine = 'Cafe' AND area = 'Sitapura';
```

AND requires both conditions. OR accepts either condition. Parentheses make combinations clear. SQL uses `=` for equality, not JavaScript `===`. Use `<>` for not equal, `>=` for at least and `<=` for at most.

```
SELECT id, name FROM restaurants
WHERE name ILIKE '%cafe%'
ORDER BY id
LIMIT 10 OFFSET 0;
```

ILIKE is PostgreSQL case-insensitive pattern matching. Percent signs match any length of text. LIMIT caps the returned rows; OFFSET skips rows. Always use a predictable ORDER BY when paging. Later, large lists may use cursor pagination instead of large offsets.

Try it yourself

Find Oslo by its name. Then return only restaurant names sorted alphabetically. Explain how changing SELECT differs from changing WHERE.

Categories and foreign keys

```
CREATE TABLE menu_categories (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  restaurant_id BIGINT NOT NULL REFERENCES restaurants(id),  
  name TEXT NOT NULL,  
  UNIQUE (restaurant_id, name)  
);
```

Each category gets its own primary key. `restaurant_id` is a foreign key to `restaurants.id`. Many category rows may share a restaurant ID because one restaurant has many categories.

`REFERENCES` rejects an insert pointing to a nonexistent restaurant. Merely naming a column `restaurant_id` would not enforce that relationship.

`UNIQUE (restaurant_id, name)` checks the pair. Student cafe can have Drinks and Snacks. Another restaurant can also have Drinks. Two exact copies of Drinks for the same restaurant are rejected. This rule does not automatically make names case-insensitive.

```
INSERT INTO menu_categories (restaurant_id, name)  
VALUES (1, 'Drinks'), (1, 'Snacks'), (2, 'Main Course')  
RETURNING id, restaurant_id, name;
```

In a fresh practice database those restaurant IDs normally match the first two inserts. Verify them first. In your project, categories already have IDs 4, 5 and 6; practice IDs can be different.

A foreign key also normally prevents deleting a restaurant while its categories still reference it. We have not enabled automatic cascading deletes.

Try it yourself

Read the generated category IDs. Explain why the category ID and restaurant ID identify different things.

Items and paise prices

```
CREATE TABLE menu_items (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  category_id BIGINT NOT NULL REFERENCES menu_categories(id),  
  name TEXT NOT NULL,  
  price_paise INTEGER NOT NULL CHECK (price_paise >= 0),  
  is_available BOOLEAN NOT NULL DEFAULT TRUE  
);
```

`category_id` links each item to one existing category. That category links to a restaurant, so the item already has a path to its restaurant without repeating restaurant details.

`price_paise` is a required whole number. CHECK rejects negative values. Our example permits a zero price; whether free items are allowed is a product decision. INTEGER has a finite maximum, so validation will also enforce reasonable application price limits.

`is_available` is required TRUE or FALSE. DEFAULT TRUE is used when the insert omits availability. `psql` may display true as t and false as f.

```
INSERT INTO menu_items (category_id, name, price_paise)  
VALUES (1, 'Tea', 3000), (1, 'Coffee', 5000)  
RETURNING id, name, is_available;
```

This example assumes Drinks has category ID 1 in practice. Replace it with the ID your previous insert returned. Your project Drinks category is 4. Leave out the generated item ID.

Try it yourself

Add an item under Snacks with the actual practice category ID. Read the rows and explain both ID columns. Convert its paise price to rupees on paper.

Why we use separate tables

A category is a thing you may create, rename, sort or hide independently of an item. Separate tables let you represent that category even before it contains any food.

Design	What happens when Drinks becomes Beverages
One table repeating names	Update the category text in every Tea, Coffee and other drink row. Miss one row and names disagree.
Separate category table	Rename the one category row. Items keep the same category_id and refer to the updated name.

The same applies to restaurant details. Saving the restaurant area on every item repeats a fact that already belongs to the restaurant.

This approach is called normalization: keep each fact in a suitable place and connect related records using IDs. It reduces inconsistent updates. It is a starting design, not a rule that every future value must be in a separate table.

```
restaurants.id
|
menu_categories.restaurant_id
menu_categories.id
|
menu_items.category_id
```

Order items will later deliberately copy an item name and price as historical snapshots. That way an old receipt does not change when a menu is edited. Repetition can be intentional when it serves a different business fact.

Try it yourself

Explain what happens to the items if you rename a category. Explain why deleting the last item should not automatically erase the category.

Join categories to items

```
SELECT
  c.name AS category_name,
  i.name AS item_name,
  i.price_paise
FROM menu_categories AS c
JOIN menu_items AS i ON i.category_id = c.id
WHERE c.restaurant_id = 1
ORDER BY c.id, i.id;
```

FROM menu_categories AS c gives that table a short name. JOIN menu_items AS i adds matching item rows. ON explains the match: an item category_id must equal the category id.

WHERE keeps categories from restaurant 1. The SELECT lines choose output columns. AS category_name labels the output so the two name columns are easy to distinguish. ORDER BY groups the display by category ID, then item ID.

Tea and Coffee both match the same Drinks category. The result repeats Drinks once for each matching item. JOIN creates a query result; it does not permanently merge or change the tables.

JOIN here means INNER JOIN: only matching pairs appear. A category with no items is absent. You already saw Tea, Coffee and Sandwich for Student cafe; Thali was absent because its category belongs to Oslo.

Try it yourself

Change only the restaurant filter to the ID for Oslo. Predict which rows appear. Run it and compare the result.

Left joins and missing values

```
SELECT c.name AS category_name, i.name AS item_name
FROM menu_categories AS c
LEFT JOIN menu_items AS i ON i.category_id = c.id
WHERE c.restaurant_id = 1
ORDER BY c.id, i.id;
```

LEFT JOIN preserves all matching restaurant categories from the left table, including empty categories. When there is no item, `item_name` is NULL. INNER JOIN would omit that category entirely.

```
SELECT c.name, COALESCE(i.name, 'No items yet') AS item_name
FROM menu_categories AS c
LEFT JOIN menu_items AS i ON i.category_id = c.id
WHERE c.restaurant_id = 1;
```

COALESCE chooses the first non-null value. It supplies a display label in this example without changing stored data.

```
SELECT name FROM menu_items WHERE name IS NULL;
```

Use IS NULL or IS NOT NULL for missing values. `name = NULL` does not test for missing values. This query should return no rows because `name` is required in our table.

Putting a condition on item columns in WHERE can remove the preserved empty categories. If you need all categories but only available matching items, include `i.is_available = TRUE` in the ON condition.

Try it yourself

Create an empty practice category. Compare INNER JOIN and LEFT JOIN results. Explain the blank item cell without guessing that the category is broken.

Update values and archive items

```
BEGIN;
```

```
UPDATE menu_items  
SET is_available = FALSE  
WHERE id = 1  
RETURNING id, name, is_available;
```

```
ROLLBACK;
```

BEGIN starts a transaction: changes can be grouped and either saved or undone. UPDATE chooses the table. SET chooses the value to change. WHERE selects which rows. RETURNING displays the changed rows. ROLLBACK undoes this practice change.

Without WHERE, UPDATE changes every row in the table. First run a SELECT with the same filter to confirm the intended rows. A price edit changes the current menu, not the saved price snapshot of an old order.

```
ALTER TABLE menu_items  
ADD COLUMN archived_at TIMESTAMPTZ;
```

ALTER TABLE changes a definition. ADD COLUMN adds optional archive information. Existing rows get NULL here. We can later archive an item instead of deleting it, keeping order references intact.

```
UPDATE menu_items  
SET archived_at = CURRENT_TIMESTAMP, is_available = FALSE  
WHERE id = 1  
RETURNING id, archived_at;
```

CURRENT_TIMESTAMP provides a timestamp for the current transaction. This final example saves normally if no transaction is open. Run it only in practice. Archiving and temporarily being unavailable are different business states.

Try it yourself

Change one practice price inside BEGIN and inspect it, then ROLLBACK. Read it again to confirm the original price is restored.

Delete deliberately and change rules

```
BEGIN;  
DELETE FROM menu_items  
WHERE name = 'Practice-only item'  
RETURNING id, name;  
ROLLBACK;
```

DELETE removes matching rows. RETURNING shows which rows were affected. ROLLBACK cancels the example. Without WHERE, DELETE removes every row in the table. Referenced rows may be protected by foreign keys.

Improve a definition

```
ALTER TABLE menu_items  
ADD CONSTRAINT menu_items_name_not_blank  
CHECK (length(trim(name)) > 0);
```

ADD CONSTRAINT gives a rule a name. trim removes ordinary leading and trailing spaces. length counts the remaining characters. This rejects an empty or space-only name. Existing rows must satisfy the new rule or adding it fails.

```
ALTER TABLE menu_items  
RENAME COLUMN is_availabe TO is_available;
```

This is the spelling fix you already performed. Do not rerun it on a corrected table; the old column no longer exists. ALTER can also change defaults or types, but changing types requires checking existing data.

DROP TABLE removes a table definition and its data. DROP DATABASE removes an entire database. TRUNCATE empties a table. You do not need these for normal restaurant operations; understand their scope before using them.

Try it yourself

In practice, attempt an empty name after adding the rule. Read the constraint error and explain which input caused it.

Count and summarize data

```
SELECT c.id, c.name, COUNT(i.id) AS item_count
FROM menu_categories AS c
LEFT JOIN menu_items AS i ON i.category_id = c.id
WHERE c.restaurant_id = 1
GROUP BY c.id, c.name
ORDER BY c.id;
```

GROUP BY collects matching rows into one group per category. COUNT(i.id) counts actual item IDs, so an empty category reports zero. COUNT(*) would count the preserved category row from the LEFT JOIN, which is different.

```
SELECT MIN(price_paise) AS cheapest_paise,
       MAX(price_paise) AS highest_paise
FROM menu_items;
```

MIN returns the smallest value; MAX returns the largest. SUM adds values and AVG calculates an average. COUNT counts rows or non-null values depending on its argument. Most aggregates return NULL for no rows; COUNT returns zero.

Later an order total will sum quantity times the saved unit price in each order item. Summing current menu prices cannot calculate historical sales correctly.

WHERE filters rows before grouping. HAVING filters groups after aggregates are calculated. Start with simple grouped queries before adding HAVING.

Try it yourself

Count available items. Then count items in each category, including an empty one. Explain why the selected count argument matters.

Transactions and concurrent orders

```
BEGIN;  
UPDATE menu_items  
SET price_paise = 3500 WHERE id = 1;  
SELECT id, price_paise FROM menu_items WHERE id = 1;  
ROLLBACK;
```

A transaction groups related changes. COMMIT saves them together. ROLLBACK cancels them. Without an explicit transaction block, individual successful SQL statements normally commit automatically.

For future order creation, the order row, order items and order event must succeed together. Saving an order without its items would leave incomplete business data. We will design that operation as one transaction.

```
BEGIN;  
SELECT id, price_paise FROM menu_items  
WHERE id = 1 FOR UPDATE;  
ROLLBACK;
```

FOR UPDATE locks the selected row against conflicting changes until the transaction ends. Another transaction may wait. A lock is for a concrete consistency requirement, not something to add to every query.

PostgreSQL defaults to Read Committed isolation. Concurrent requests can still produce races unless the operation, constraints and locking are designed correctly. Deadlocks or serialization failures may require bounded retries later.

In Node, a transaction must use the same checked-out database client for BEGIN, every query, and COMMIT or ROLLBACK. Separate pool.query calls can use different connections, so they cannot form a reliable transaction.

Try it yourself

Run the price change inside a transaction, then cancel it. Explain why an order and its order items must be saved together.

Indexes and query plans

```
CREATE INDEX menu_items_category_id_idx  
ON menu_items (category_id);
```

An index is an extra lookup structure. It can help PostgreSQL find items for a category without checking every row. It uses storage and adds work to inserts and updates. More indexes are not automatically better.

PRIMARY KEY and UNIQUE constraints already create supporting indexes. A foreign key does not automatically create an index on the referencing column, so we may add one for common lookups.

```
EXPLAIN  
SELECT id, name FROM menu_items WHERE category_id = 1;
```

EXPLAIN shows the planned work without running this SELECT. With only a few rows, PostgreSQL may choose a sequential scan even with an index; reading the tiny table can be cheaper.

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT id, name FROM menu_items WHERE category_id = 1;
```

ANALYZE here actually executes the statement and reports timing and actual row counts. Use it thoughtfully: on UPDATE or DELETE it would perform the write. This example is a read.

For future restaurant order queues, index the columns used in restaurant-scoped filters and sorting after measuring the actual queries. Do not guess performance needs before a useful query exists.

Try it yourself

Compare the plans before and after the index. If PostgreSQL still scans a tiny table, explain why that can be reasonable.

SQL files and database migrations

So far you typed commands interactively. Save future schema changes in ordered SQL files so another machine can reproduce the database and Git can record its history. A migration is one versioned schema change.

```
-- Example contents of 004_add_item_description.sql
ALTER TABLE menu_items
ADD COLUMN description TEXT;
```

The SQL comment describes the file; the statement adds an optional column. Put this in a file you create yourself. A migration tool later tracks which files have already run. `psql` by itself does not track that history.

From the shell, in the folder containing the file

```
PG_BIN=/opt/homebrew/opt/postgresql@18/bin
"$PG_BIN/psql" -d restaurant_ordering_practice \
-v ON_ERROR_STOP=1 -1 -f 004_add_item_description.sql
```

`PG_BIN` is a shell variable for the program directory. `-f` reads SQL from a file. `ON_ERROR_STOP` makes the script stop at the first error. `-1` runs this compatible script as one transaction. Some commands, including `CREATE DATABASE`, cannot run inside that transaction.

Inside `psql`, `\i filename.sql` also runs a file. Migration SQL, test seed data and backups have different jobs. Keep sample inserts out of production schema migrations unless they are required reference data.

Try it yourself

Create a practice SQL file adding description. Run it once and inspect the definition. Explain why running it twice fails.

Roles, permissions and tenant boundaries

Your local learning connection uses your Mac-named PostgreSQL role. Before production, the API needs its own restricted database role. Application customers and staff are separate identities managed by your backend.

```
CREATE ROLE restaurant_api LOGIN;  
\password restaurant_api  
GRANT CONNECT ON DATABASE restaurant_ordering_practice  
TO restaurant_api;  
GRANT USAGE ON SCHEMA public TO restaurant_api;  
GRANT SELECT ON restaurants, menu_categories, menu_items  
TO restaurant_api;
```

CREATE ROLE makes a database identity. LOGIN permits connections. \password prompts for a password without writing it into SQL history. GRANT CONNECT permits this database connection. USAGE permits schema access; SELECT permits reading the listed tables. It grants neither INSERT nor ALTER privileges.

This is an illustrative read-only role, not the finished production permission setup. Ownership, PUBLIC defaults, future-table privileges and connection authentication must also be reviewed. Successful permission grants do not themselves require password authentication.

Database roles do not automatically separate Restaurant A from Restaurant B when both use the same API role. Every API lookup and write must enforce the acting user membership and restaurant scope. Optional row-level security can add another layer later.

For future cross-restaurant relationships, constraints must ensure linked records belong to the same restaurant. Our simple item model derives restaurant ownership through the category instead of storing contradictory restaurant IDs.

Try it yourself

Explain why knowing a restaurant ID is not permission to edit that restaurant. Keep secret passwords out of source code and screenshots.

Backups and restoring to a new database

A backup is a saved copy that you have actually tested restoring. Practice with a separate destination database. The example below does not overwrite the project database.

In the shell

```
PG_BIN=/opt/homebrew/opt/postgresql@18/bin
"$PG_BIN/pg_dump" -d restaurant_ordering_practice \
  -Fc -f restaurant_practice.dump
"$PG_BIN/createdb" restaurant_ordering_restore_test
"$PG_BIN/pg_restore" --no-owner --no-privileges \
  --exit-on-error -d restaurant_ordering_restore_test \
  restaurant_practice.dump
```

`pg_dump` reads one database and creates a backup. `-Fc` chooses its custom archive format. `-f` names the output file. `createdb` creates a new empty destination; pick another name if it already exists.

`pg_restore` loads that archive into the destination. `--no-owner` and `--no-privileges` omit original ownership and grants for this local practice restore. They are not a complete production permission restoration policy. `--exit-on-error` stops on failure.

Connect to the restored database and inspect the tables and rows. Compare counts with the source. A file existing on disk is not proof that recovery works.

`pg_dump` does not include server-wide roles; those require separate planning. Production recovery also needs retention, off-machine storage, access protection, and a chosen recovery target. We will learn those after the local restore exercise.

Try it yourself

Restore your practice database to a new database. Verify menu items and category foreign keys exist. Keep the source database unchanged.

How Express uses the database

Your db.js

```
const { Pool } = require('pg')
const pool = new Pool()
module.exports = pool
```

require loads the pg package. The braces take its Pool constructor. new Pool creates a manager for reusable database connections; it opens connections when needed. It does not store restaurant rows. module.exports shares this instance with other files.

Your route handler uses the pool

```
const pool = require('./db')

// Inside an async route handler:
const result = await pool.query(
  'SELECT id, name FROM restaurants WHERE id = $1',
  [id]
)
res.status(200).json(result.rows[0])
```

require loads the exported pool. query sends SQL and a separate values array. \$1 represents the first array value. Parameters are values, not executable SQL text; they do not replace column or table names. Keep validation and authorization as well.

query returns a Promise, a result that will arrive later. async permits await in the handler. await waits for that result while other requests can run. result.rows is an array of row objects; rows[0] is the first object. Check for an empty array before sending it.

PostgreSQL BIGINT IDs arrive as strings by default in pg, preserving large values accurately. The API returns a JSON response, not the whole database connection.

```
PGDATABASE=restaurant_ordering_dev npx nodemon index.js
```

This shell command selects the database through an environment variable. The pool also accepts host, port, user and password settings. Our current local defaults are for learning; deployment needs explicit restricted credentials.

Learning checkpoints and references

Stop building new features until you can explain the existing three tables and their relationships. Work through a few small examples each day; the guide is a reference you can return to, not a test of memory.

Checkpoint	What you should be able to explain
After lessons 2-8	Where commands run; a table versus a row; types; insert and read.
After lessons 9-14	Filters; relationship IDs; constraints; why a JOIN finds the right restaurant menu.
After lessons 15-18	How to change one row; how to undo a transaction; why orders need atomic writes.
After lessons 19-23	Indexes; repeatable migrations; permissions; restore checks; how Node sends queries.

Practice questions: Why can `restaurant_id` repeat in categories? What prevents an item using a missing category? Why does 3000 mean INR 30? Why can a JOIN display Drinks twice? What happens when UPDATE has no WHERE?

Official references

[PostgreSQL beginner tutorial](#)

[psql commands](#)

[Table constraints](#)

[SELECT and joins](#)

[Transactions](#)

[Indexes](#)

[Backup and restore](#)

[Node PostgreSQL queries](#)

[Node PostgreSQL transactions](#)

Later project topics include order snapshots, idempotency constraints, QR session expiry, joins across tenant-scoped orders, audit records, and pagination. We will introduce their exact schema and queries when the requirements are concrete. PostgreSQL has many additional commands; you do not need all of them to begin this project.

The commands your full project needs

I reread your original project guide. Its product scope is dine-in QR or PIN ordering, restaurant staff operations, manual cash or UPI confirmation, monthly or yearly subscriptions and an admin console. Your current choices replace its Firebase and Firestore stack with local PostgreSQL and Express; those choices control this learning guide.

Customers enter a required display name. Direct QR visitors validate table access on entry; homepage visitors validate it at checkout. Customer sign-in is optional. Staff and owners require verified identities and permissions.

Feature	Database skills it uses
Restaurant discovery	SELECT, WHERE, ILIKE, ORDER BY, pagination.
Menus and options	Foreign keys, JOIN, INSERT, UPDATE, archive columns.
Tables and QR sessions	Unique tokens, timestamps, expiry filters, revocation.
Staff permissions	Membership queries and restaurant-scoped lookups.
Orders and payments	Transactions, snapshots, checks, conditional UPDATE, audit inserts.
Subscriptions	Time comparisons, INTERVAL, conditional access rules.
Operations	Migrations, indexes, roles, backups and restore checks.

The following lessons are future practice examples, not commands already used in your project. They introduce SQL for the full feature set. The final production schema still needs design reviews and tests; SQL cannot replace authorization, rate limiting, OAuth verification or payment verification in Express.

Try it yourself

Name the feature before running a query. Ask: am I reading, adding, changing or deleting data? Which restaurant should this operation belong to?

Create a separate lab for future features

```
CREATE SCHEMA project_lab;  
SET search_path TO project_lab, public;  
SHOW search_path;
```

Run these in `restaurant_ordering_practice` only. `CREATE SCHEMA` creates a namespace. `SET search_path` tells this connection where to resolve unqualified table names and create new tables. `SHOW` displays the setting. Reapply `SET` after opening a new connection.

Future examples use this lab and reference the practice `public.restaurants` and `public.menu_items` tables you already built. They do not add orders or authentication tables to your real project. Verify the referenced IDs before inserting example data.

```
CREATE TABLE app_users (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  display_name TEXT NOT NULL CHECK (length(trim(display_name)) > 0)  
);  
  
CREATE TABLE auth_identities (  
  user_id BIGINT NOT NULL REFERENCES app_users(id),  
  provider TEXT NOT NULL,  
  provider_subject TEXT NOT NULL,  
  PRIMARY KEY (provider, provider_subject)  
);
```

`app_users` gives an internal identity. A display name is a label, not a login secret. `auth_identities` maps a verified external provider identity to it. The combined primary key prevents linking the same provider account to two users. `provider_subject` is the provider stable account ID, not a self-submitted email.

OAuth login must be verified in Express using the provider protocol and appropriate session security. SQL only stores the resulting association. Never store a plain password as a display name or token.

Try it yourself

Explain why two users can have the same display name while their internal IDs differ. Guest order access will use sessions without requiring this account.

Restaurant membership and permissions

```
CREATE TABLE restaurant_members (  
  restaurant_id BIGINT NOT NULL REFERENCES public.restaurants(id),  
  user_id BIGINT NOT NULL REFERENCES app_users(id),  
  role TEXT NOT NULL CHECK (role IN  
    ('owner', 'manager', 'cashier', 'kitchen', 'waiter')),  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  PRIMARY KEY (restaurant_id, user_id)  
);
```

The pair identifies one user membership in one restaurant. IN means the value must be one of the listed role names. The same user can have a membership in another restaurant. This lab allows one role per restaurant; multiple roles would require a different design.

```
SELECT role FROM restaurant_members  
WHERE restaurant_id = 1 AND user_id = 1 AND is_active = TRUE;
```

Use the verified user ID from the server session and the restaurant being accessed. Express translates the returned role into allowed actions. An empty result means there is no active membership; it is not permission to proceed.

In a Node query these values become placeholders. A platform administrator needs separate platform permissions; that role should not be accepted as a restaurant membership supplied by the client.

A SQL CHECK permits role names; it does not decide who is allowed to assign them. Express must prevent a cashier from promoting themselves, and a manager from creating an owner when the policy forbids it.

Try it yourself

Explain authentication as proving an identity and authorization as checking an action. Membership connects an authenticated staff identity to a particular restaurant.

Menu options and consistent ownership

```
CREATE TABLE menu_item_options (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  menu_item_id BIGINT NOT NULL REFERENCES public.menu_items(id),
  name TEXT NOT NULL,
  extra_price_paise INTEGER NOT NULL CHECK (extra_price_paise >= 0),
  is_available BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (menu_item_id, name)
);
```

This simple lab models add-ons such as extra cheese. `menu_item_id` ties the option to an item, and the item ties to a category and restaurant. Required choices, multiple option groups and selection limits need more tables and business rules later.

```
SELECT i.id, i.name, i.price_paise
FROM public.menu_items AS i
JOIN public.menu_categories AS c ON c.id = i.category_id
WHERE c.restaurant_id = 1 AND i.is_available = TRUE;
```

A restaurant menu query follows the category relationship and filters by restaurant. Future queries must also exclude archived items after the archive migration. Availability and publication must be checked when creating an order, not just when displaying the menu.

If a customer submits an option ID, the server must verify it belongs to the selected item. Joining by relationships avoids accepting an option from another restaurant.

The server reads current item and option prices and calculates the order amount. A cart may display a price for convenience, but that client price is not trusted input for payment or order storage.

Try it yourself

Describe the relationship chain from a cheese option to its item, category and restaurant. Explain why the option price is stored separately from the item base price.

Restaurant tables and QR tokens

```
CREATE TABLE restaurant_tables (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  restaurant_id BIGINT NOT NULL REFERENCES public.restaurants(id),  
  table_number INTEGER NOT NULL CHECK (table_number > 0),  
  qr_token_hash TEXT NOT NULL UNIQUE,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  UNIQUE (restaurant_id, table_number),  
  UNIQUE (id, restaurant_id)  
);
```

Each physical table has its own record. Table number 1 can exist in different restaurants; the combined UNIQUE rule prevents two table 1 records in the same restaurant. The last UNIQUE pair supports the composite foreign key in the next lesson.

Express generates a long random QR token and stores a hash of it here. The printed QR contains the link and raw token. The hash is not a table number, customer identity, or proof that a person is physically present.

```
SELECT id, restaurant_id, table_number  
FROM restaurant_tables  
WHERE qr_token_hash = 'EXAMPLE_HASH' AND is_active = TRUE;
```

This literal is only a teaching placeholder. The real API hashes the scanned token, uses a parameterized lookup, and checks restaurant activity and ordering access. No result means access must not be issued.

Rotation replaces the stored token hash and invalidates the old QR. PIN access needs a separate secure design: hashes, expiry and rate limits. A static shared QR can be copied; staff order acceptance and other controls address that limitation.

Try it yourself

Explain why `table_number` alone is not a secure access credential. What changes when a printed QR token is rotated?

Guest table sessions and expiry

```
CREATE TABLE table_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  restaurant_id BIGINT NOT NULL REFERENCES public.restaurants(id),
  table_id BIGINT NOT NULL,
  access_token_hash TEXT NOT NULL UNIQUE,
  display_name TEXT,
  user_id BIGINT REFERENCES app_users(id),
  expires_at TIMESTAMPTZ NOT NULL,
  revoked_at TIMESTAMPTZ,
  UNIQUE (id, restaurant_id),
  FOREIGN KEY (table_id, restaurant_id)
    REFERENCES restaurant_tables(id, restaurant_id)
);
```

UUID gives the session an identifier. `gen_random_uuid()` generates it in PostgreSQL; it is not the browser access secret. Express creates a separate random secret and stores its hash. The browser receives the secret using the chosen secure session mechanism.

`display_name` is temporarily optional because direct QR visitors validate before entering a name. The order service requires a nonblank name before ordering. `user_id` is nullable so guests can order. Optional login can associate the session with a verified user.

The combined foreign key requires both table and restaurant to match the same table record. `expires_at` ends the access period; `revoked_at` records cancellation. Keep credentials out of logs.

```
SELECT id, restaurant_id, table_id FROM table_sessions
WHERE access_token_hash = 'EXAMPLE_HASH'
  AND revoked_at IS NULL AND expires_at > CURRENT_TIMESTAMP;
```

Repeat validity checks at order creation. Express also verifies cart restaurant, active table and restaurant policy. A shared table QR must not let one guest read another guest order.

Try it yourself

Explain why knowing a table QR does not authorize all order history. Session expiry applies on the server, not only in the browser.

Order headers and safe retries

```
CREATE TABLE orders (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  restaurant_id BIGINT NOT NULL REFERENCES public.restaurants(id),
  table_session_id UUID NOT NULL,
  customer_name TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'submitted' CHECK (status IN
    ('submitted', 'accepted', 'preparing', 'ready', 'completed',
    'rejected', 'cancelled')),
  payment_status TEXT NOT NULL DEFAULT 'unpaid' CHECK
    (payment_status IN ('unpaid', 'customer_claimed', 'confirmed')),
  total_paise BIGINT NOT NULL CHECK (total_paise >= 0),
  currency TEXT NOT NULL DEFAULT 'INR' CHECK (currency = 'INR'),
  idempotency_key TEXT NOT NULL,
  request_fingerprint TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (table_session_id, idempotency_key),
  FOREIGN KEY (table_session_id, restaurant_id)
    REFERENCES table_sessions(id, restaurant_id)
);
```

The order header holds the order as a whole. Status CHECK rules permit labels; they do not enforce the sequence between labels. A conditional UPDATE will handle transitions. `customer_name` is a snapshot for staff; add nonblank validation before writes.

A retry uses the same idempotency key within the session. UNIQUE stops two rows using that pair. The request fingerprint records what was requested, so reusing a key for a different cart must be rejected. Creating order items, total, and events still needs one transaction.

The foreign key prevents a session belonging to one restaurant from being attached to an order in another. The API must also check expiry and authority; a foreign key does not do that.

Try it yourself

Why must a network retry not create a second meal order? Why is a unique key alone insufficient if a retry sends different items?

Order item snapshots and totals

```
CREATE TABLE order_items (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id),
  menu_item_id BIGINT NOT NULL REFERENCES public.menu_items(id),
  item_name TEXT NOT NULL,
  unit_price_paise BIGINT NOT NULL CHECK (unit_price_paise >= 0),
  quantity INTEGER NOT NULL CHECK (quantity BETWEEN 1 AND 99)
);
```

`order_id` links each line to the order header. `item_name` and `unit_price_paise` copy the server-verified menu details at order time. Later menu edits cannot alter these saved historical facts. The quantity bound is an example product limit; validate the same rule in Express.

Selected options will need their own snapshots too. This teaching table only covers the base item line. `menu_item_id` keeps a relationship to the original item, which should be archived rather than deleted.

```
SELECT COALESCE(SUM(quantity::bigint * unit_price_paise), 0)
  AS total_paise
FROM order_items WHERE order_id = 1;
```

SUM adds line amounts. Multiplication calculates each quantity times unit price. `::bigint` casts quantity to a larger integer before arithmetic. COALESCE supplies zero for no rows. The real service bounds line and cart totals to prevent overflow and rejects empty orders.

Two teas at 3000 paise and one coffee at 5000 total 11000 paise, or INR 110. Future option snapshots contribute to the price calculation. Tax and rounding rules require explicit design if they are introduced.

Try it yourself

Calculate the sample total on paper. Explain why a future receipt must read order snapshots rather than today menu prices.

Conflicts, retries and atomic creation

```
INSERT INTO orders
  (restaurant_id, table_session_id, customer_name, total_paise,
   idempotency_key, request_fingerprint)
VALUES (1, '00000000-0000-0000-0000-000000000001',
        'Example guest', 11000, 'example-key', 'example-hash')
ON CONFLICT (table_session_id, idempotency_key) DO NOTHING
RETURNING id;
```

This is an illustrative statement: the session UUID must be replaced with an existing authorized lab session before it can run. The values are not real credentials. Production queries use placeholders rather than these literals.

ON CONFLICT handles the named duplicate key. DO NOTHING avoids inserting a second order; RETURNING gives the new ID only if a row was inserted. No returned row requires a separate lookup of the existing order and a fingerprint comparison, using the same session scope.

Run order header creation, all item and option snapshots, and the initial event in a single transaction. If any part fails, ROLLBACK. Before COMMIT, require at least one valid line and a total consistent with those snapshots.

A transaction protects completeness; a unique constraint protects against duplicate creation. Both are needed. A menu may change during checkout, so define the consistency rule and use suitable locks or isolation when reading prices and availability.

Do not use an unconditional ON CONFLICT DO UPDATE for payment confirmation or an order retry. That could overwrite an unrelated state or change the meaning of a previously accepted request.

Try it yourself

Explain what RETURNING reports after a duplicate. Describe what the API must compare before returning an existing order to a retry.

Order transitions, payments and audits

```
CREATE TABLE order_events (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id),
  actor_user_id BIGINT REFERENCES app_users(id),
  event_type TEXT NOT NULL,
  reason TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
UPDATE orders SET status = 'preparing'
WHERE id = 1 AND restaurant_id = 1 AND status = 'accepted'
RETURNING id, status;
```

The UPDATE succeeds only for an accepted order in that restaurant. Zero returned rows means the condition was not met; do not pretend it succeeded. Express checks the staff permission before the write. State change and event insert belong in the same transaction.

Customer cancellation additionally checks the customer session and submitted state. Staff cancellations require a reason. Checking a status in a separate SELECT and then doing an unconditional UPDATE permits races; the conditional write checks the current state as it changes it.

```
UPDATE orders SET payment_status = 'customer_claimed'
WHERE id = 1 AND table_session_id =
  '00000000-0000-0000-0000-000000000001'
  AND payment_status = 'unpaid'
RETURNING id, payment_status;
```

This session UUID is a placeholder. A customer claim is not verified payment. Only authorized staff may confirm cash or restaurant UPI payment. Provider automation later requires verified, deduplicated webhooks.

A broader audit table can store restaurant, actor, action, entity and safe JSONB details. JSONB holds structured metadata; keep money, ownership and status in constrained columns. Audit records should exclude passwords, raw tokens and unnecessary personal data.

Try it yourself

Explain why a status CHECK alone does not prevent submitted from jumping straight to completed. Why must the payment claim remain separate from confirmed?

Subscriptions and scheduled expiry work

```
CREATE TABLE subscriptions (
  restaurant_id BIGINT PRIMARY KEY REFERENCES public.restaurants(id),
  status TEXT NOT NULL CHECK
    (status IN ('trial', 'active', 'grace', 'paused')),
  period_end TIMESTAMPTZ NOT NULL,
  grace_until TIMESTAMPTZ NOT NULL,
  CHECK (grace_until >= period_end)
);
```

```
SELECT CURRENT_TIMESTAMP + INTERVAL '7 days' AS grace_until;
```

One subscription row describes the current restaurant entitlement in this lab. Billing events and history need separate records later. `INTERVAL` represents a duration; adding it to a timestamp calculates another timestamp. The guide specifies seven days of grace.

```
SELECT restaurant_id FROM subscriptions
WHERE restaurant_id = 1 AND
  ((status IN ('trial', 'active') AND period_end > CURRENT_TIMESTAMP)
   OR (status = 'grace' AND grace_until > CURRENT_TIMESTAMP));
```

This example checks ordering entitlement using status and timestamps. The real API checks it when creating an order. A delayed scheduled job must not allow new orders just because it has not changed a stale status yet.

```
UPDATE subscriptions SET status = 'paused'
WHERE status = 'grace' AND grace_until <= CURRENT_TIMESTAMP
RETURNING restaurant_id;
```

A background job may update expired states. Account for trial and active expiry paths too. Pausing stops new orders but does not prevent staff finishing existing orders or exporting records, as your project guide requires.

Date filtering for restaurant daily totals needs a chosen time zone. Use explicit start and end instants with a half-open range: `created_at >= start` and `created_at < end`. Sum confirmed-payment records or completed sales according to a clearly defined report; those are not necessarily the same number.

Try it yourself

Explain why comparing time in Express alone is insufficient if the database policy depends on stored timestamps. Which actions should remain allowed after a subscription pauses?

Admin changes, files and production tools

Admin onboarding is a transaction creating restaurant settings, an owner membership and subscription state. Suspension is a controlled UPDATE with an audit insert. Approved UPI details need a separate approval record; customer input cannot overwrite them.

```
CREATE TABLE audit_logs (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  restaurant_id BIGINT REFERENCES public.restaurants(id),
  actor_user_id BIGINT REFERENCES app_users(id),
  action TEXT NOT NULL,
  details JSONB NOT NULL DEFAULT '{} '::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

JSONB stores structured details. The empty object default is converted to JSONB using `::jsonb`. In Node, pass metadata as a query parameter; do not concatenate user text into an audit INSERT. Append-only permissions and retention rules come later.

For uploads, store image metadata such as `storage_key`, media type, byte size and restaurant ownership. The file itself can live in a local protected upload directory initially. SQL does not check file safety or serve image URLs; Express handles validation and access.

```
CREATE INDEX orders_restaurant_queue_idx
ON orders (restaurant_id, status, created_at, id);

CREATE INDEX table_sessions_expiry_idx
ON table_sessions (expires_at);
```

These are candidate indexes for a restaurant queue and expiry work. Measure real query plans before adopting them. A queue query still needs restaurant filtering and authorization; indexes do not enforce access.

Later tools include `ANALYZE` for planner statistics, `VACUUM` for routine row cleanup, `statement_timeout` and `lock_timeout` for bounded waits, and `pg_stat_activity` for observing connections. Autovacuum normally handles routine cleanup. Learn these during operations work; do not start by running aggressive maintenance commands.

Try it yourself

Choose the database operation for an admin suspension and explain which additional audit and permission checks it needs.

Your study path and command reference

Read lessons 2-14 first. Rebuild the three small tables in practice and explain every field out loud. Continue to changes and transactions once you understand those relationships. Future feature lessons explain SQL building blocks; introduce them in the project only as each feature is designed.

Goal	Commands to find in this guide
Get connected	brew services, pg_isready, psql, \c, \conninfo.
Inspect	\l, \dt, \d, SELECT, SHOW.
Model	CREATE DATABASE, CREATE SCHEMA, CREATE TABLE, constraints.
Read	WHERE, AND, OR, IN, ILIKE, JOIN, ORDER BY, LIMIT.
Write	INSERT, RETURNING, UPDATE, DELETE, ON CONFLICT.
Keep changes together	BEGIN, COMMIT, ROLLBACK, FOR UPDATE.
Improve and reproduce	ALTER TABLE, CREATE INDEX, EXPLAIN, SQL files.
Operate	CREATE ROLE, GRANT, pg_dump, pg_restore.

Not every possible future SQL statement can be known before the final design. This guide covers the current commands and the common toolbox for every feature in your stated MVP. Stored procedures, triggers, partitioning and replication can wait until a real requirement justifies them.

Project reference and documentation

Source scope: restaurant-ordering-project-guide.docx, provided by you. Product requirements were retained; Firebase-specific instructions were adapted to your explicit local PostgreSQL and Express choices.

[PostgreSQL tutorial](#)

[All SQL command reference](#)

[psql reference](#)

[Constraints](#)

[Node queries and transactions](#)